OCA → openAG tool mapping

What each `opencode-arch` (OCA) `arch_*` MCP tool delegates to in the openAG engine, and what OCA adds on top.

> Generated from source at commit `e797f3a`. If the tool surface changes, regenerate this doc and its PDF (`pandoc docs/openag-mapping.md -o docs/openag-mapping.pdf --pdf-engine=xelatex`).

Thesis

openAG is the kernel. It holds the algorithms and domain logic — pure functions over in-memory docs and graphs (`ModelGraph`, the B-ROUND fixpoint, generations, mining, lifecycle, AI jobs). It mandates no file paths and exposes no transport.

OCA is the delivery shell. It adds five cross-cutting concerns and **almost no domain logic**:

1. **MCP surface** — each tool is a `@mcp.tool()` in `mcp/server.py` exposing an openAG function over the protocol with a typed signature + docstring.
2. **On-disk persistence contract** — the "write-once-then-read" convention openAG itself does not define: writers save to `.architecture/model/model.v3.yaml` (`save_model_dir`) + `.architecture/generations/openAG-tool` (`GenerationStore`); readers rehydrate via `_openag_loader.load_doc` (merge `*.v3.yaml` fragments → `ModelGraph.from_doc`).
3. **Uniform envelope** — `mcp/envelope.py:tool_result` normalizes every return to `{ok, error:{code,message,details}}` with an exception→code taxonomy.
4. **Proprioception + telemetry** — `mcp/_arch_tool.py:arch_tool` emits `in/out` spans (`component_id=tool:<name>`) and appends a per-invocation record to `.architecture/reservoir/tool_telemetry.jsonl`. All fail-soft.
5. **Repo resolution + legacy shims** — `resolve_repo` validation on every call; AMS-era params (`model_yaml`, `stage`, `resolutions`, …) are accepted-and-ignored for call-site compatibility.

Every tool = **universal `@arch_tool` layer** (all five above) + one or more openAG calls. The "OCA adds on top" column in the tables below describes what is added *beyond* that universal layer.

The universal layer

Applied to all ~60 tools via the `@arch_tool` decorator (`mcp/_arch_tool.py`), which composes `tool_result` (`mcp/envelope.py`):

Concern	Mechanism	Effect
Repo resolution	`resolve_repo(repo_path)`	expand/resolve path, validate it exists and is a dir
Envelope normalization	`tool_result`	returns + exceptions → `{ok, error:{code,message,details}}`
Error taxonomy	exception → code map	`ValueError`→`INVALID_ARGUMENT`, `FileNotFoundError`→`NOT_FOUND`, `jsonschema.ValidationError`→`SCHEMA_VIOLATION`, else `INTERNAL`
Proprioceptive spans	`openag.observe.decorator.emit`	`in/out` spans tagged `component_id=tool:<name>`, `io_kind=tool_call`
Mining telemetry	`openag.mining.reservoir.append_records`	one append-only record per call → `.architecture/reservoir/tool_telemetry.jsonl`

Concern	Mechanism	Effect
Fail-soft side effects	try/except around bind+emit+telemetry	no side-effect error can alter the returned envelope

Error codes (`mcp/envelope.py`): `NOT_FOUND`, `INVALID_ARGUMENT`, `SCHEMA_VIOLATION`, `PRECONDITION_FAILED`, `INTERNAL`.

The persistence contract

openAG functions take/return docs and store roots as plain arguments. OCA pins them to a fixed tree in the target repo:

```
.architecture/
├── model/
│   └── model.v3.yaml        # merged by load_doc; written by save_model_dir
├── generations/
│   └── openAG-tool/         # GenerationStore monotone snapshots
└── reservoir/
    └── tool_telemetry.jsonl # per-invocation telemetry (universal layer)
```

- **Readers** rehydrate the model via `_openag_loader.load_doc`, which merges the `*.v3.yaml` fragments and raises `FileNotFoundError: openAG model dir not found/empty` if no writer has run.
- **Three tools scan source fresh** instead of (or in addition to) reading the persisted model: `arch_pipeline`, `arch_check`, `arch_gate` (all via `graph.ground.scan_ground_graph`).
- `arch_scan` is the only pure reader that needs **no** model at all.

Per-tool mapping

Writers — scan source → persist model + generation

OCA tool	openAG backend	OCA adds on top
`arch_pipeline`	`graph.ground.scan_ground_graph` + `extract.pipeline._seed_doc` + `kernel.pipeline.run_pipeline` (→`b_round`)	orchestrates scan→seed→fixpoint; persists model dir + generation; shapes grounding summary
`arch_extract`	`extract.pipeline.extract_model` (→`b_round`)	persists model dir + generation; returns entity/relationship counts
`arch_author`	`llm.author.author_from_requirements` + `kernel.generations`	persists a generation authored from requirements text
`arch_ingest`	`graph.ingest.ingest_source_graph`	wraps SourceGraph-JSON → deterministic doc
`arch_grow`	`kernel.forward_grow` + `kernel.gap_report.grounding_residue` + `kernel.provider_factory`	loads persisted model; scoped forward-grow; provider wiring

Scan / inspect — readers (load persisted model unless noted)

OCA tool	openAG backend	OCA adds on top
`arch_scan`	`graph.scan_report.scan_report`	fresh source scan — **no model dir needed**

OCA tool	openAG backend	OCA adds on top
`arch_frontier` / `arch_owed` / `arch_gaps` / `arch_slice` (`arch_context.py`)	`kernel.readiness.ready_frontier`, `kernel.promise_engine.owed_promises_graph`, `kernel.gate.emit_gaps_graph`, `graph.slicer.slice_by_entity`	`load_doc`→graph; `node_id` scoping
`arch_health`	`kernel.health.graph_health` / `node_health` + `observe.journal`	`load_doc`; merges runtime anomaly flags
`arch_round`	`kernel.round.b_round` + `graph.slicer`	re-runs fixpoint on the persisted model; node scope
`arch_decompose`	`kernel.decompose_report.decompose_report`	`load_doc`→`ModelGraph`
`arch_group`	`kernel.grouping.group_modules`	`load_doc`
`arch_compliance`	`compliance.matrix` / `axes_report` / `scoring` + `kernel.gate`	`load_doc`; report-mode switch

Validate / gate / evaluate

OCA tool	openAG backend	OCA adds on top
`arch_validate`	`validate.validate_scored`	`load_doc` **or** inline `model_yaml`
`arch_check`	`kernel.representativeness.check_representativeness` + `scan_ground_graph` (**fresh**)	persisted model vs freshly-scanned ground graph
`arch_gate`	`kernel.gate.gate_report` + `scan_ground_graph` (**fresh**)	`load_doc` + fresh scan for completeness gaps
`arch_evaluate`	`kernel.evaluate.evaluate_workspace`	whole-workspace rollup over generations
`arch_llm_audit`	`kernel.llm_audit.audit_decomposition`	`load_doc`; deterministic second opinion

Change / history / identity

OCA tool	openAG backend	OCA adds on top
`arch_diff`	`model_io.load_model_dir` + `diff.doc_diff.diff_docs`	resolves two docs (default git-HEAD baseline)
`arch_history`	`kernel.generations.GenerationStore` (via `_generations`)	timeline / node-trajectory / gen-diff projection
`arch_bind`, `arch_commit_generation` (`arch_generation.py`)	`kernel.identity.crosswalk` / `identity_gaps` + `kernel.health.graph_health`	`load_doc`; theta-crosswalk / monotone commit

Docs / export / views / artifacts

OCA tool	openAG backend
`arch_docs`	`viewer.docs.generate_docs`
`arch_export`	`graph.export_bundle.export_bundle`
`arch_view_project` / `arch_view_render`	`lifecycle.views.project_view` / `render_view`
`arch_slice_materialize`	`lifecycle.slices.materialize_slice`
`arch_artifact_plan` / `arch_artifact_rebuild`	`lifecycle.artifacts` (plan / `rebuild` / `ArtifactCycle`)

Requirements / feedback / stats

OCA tool	openAG backend
arch_require	compliance.requirements.capture_requirement
arch_trace_requirements	compliance.trace.trace_requirements
arch_correct	mining.corrections.store_correction
arch_feedback	mining.feedback.record_feedback
arch_stats	mining.stats.aggregate_stats

Decomposition primitives — pure; mostly no repo/model

OCA tool	openAG backend
arch_propose_cut / arch_score_cut / arch_allocate_contract (arch_cut.py)	llm.proposal + kernel.round._split + graph.cut_score.cut / score_cut + kernel.contracts.allocate_contract
arch_integrate	kernel.integrate.integrate + kernel.contracts.ChildContract

Lifecycle / packaging — published generations

OCA tool	openAG backend
arch_package_publish / arch_package_load / arch_package_list_generations / arch_package_diff	lifecycle.package.*
arch_package_merge	lifecycle.merge.three_way_merge + GenerationStore
arch_package_stale	lifecycle.staleness.stale_report
arch_package_children_add	lifecycle.children.add_child
arch_proposal_validate / arch_proposal_apply	lifecycle.proposals.validate_proposal_doc / apply_proposal_doc

AI jobs / work / learning

OCA tool	openAG backend	OCA adds on top
arch_workorder_submit	ai.work_order.WorkOrder + ai.jobs.JobStore	persists WorkOrder + draft Job + journals submission
arch_job_get / arch_job_transition / arch_job_run	ai.jobs.JobStore / JobState + ai.worker.run_job	state-machine CRUD over the protocol
arch_work_issue	ai.work_order + ai.jobs + ai.worker.run_job + lifecycle.journal	full comment→issue→dev ceremony loop
arch_lessons	kernel.lessons + mining.reservoir	mined lessons + reviewable stubs
arch_learn / arch_learn_approve	kernel.lessons + ai.jobs + ai.worker.run_job	materialize / release a learning WorkOrder
arch_mine	mining.runner.mine_run + mining.sources + provider_factory	reservoir mining over OpenCode sessions

Proprioception — runtime observation

OCA tool	openAG backend
arch_flags	observe.journal + observe.audit.audit_observations
arch_observe	observe.journal (filtered records)

OCA tool	openAG backend
`arch_ingest_telemetry`	`observe.event.Observation` (append events)

Structural notes

- **Fresh-scan tools:** `arch_pipeline`, `arch_check`, `arch_gate` scan source directly via `scan_ground_graph`; everything else reads the persisted `.architecture/model/`.
- **Convention ownership:** the `.architecture/{model,generations,reservoir}/` layout is entirely an OCA convention. openAG's functions take docs and store roots as plain arguments; OCA pins them to that tree.
- **No domain logic in OCA:** every row's compute lives in openAG. OCA's value is the MCP surface, the persistence contract, the uniform envelope, the telemetry/proprioception, and backward-compatible call sites.